

Web VAPT Section 13 - Evidence Capture, Redaction, and GitHub Reporting

Professional documentation for screenshot evidence handling, safe portfolio publishing, and reporting discipline.

Enterprise Security Assessment Lab

PROJECT AREA	Web Application Vulnerability Assessment and Penetration Testing
PRIMARY TARGET	DVWA running in local lab environment
TESTING ENVIRONMENT	Kali Linux attack machine with Docker-based vulnerable target
PREPARED BY	Jagriti Banerjee
DOCUMENT TYPE	Individual Web VAPT Attack-Section PDF

AUTHORIZED LAB TESTING ONLY

No live client data. No production systems. Screenshots and outputs must be redacted before public upload.

1. Document Purpose

This individual PDF documents the Web VAPT attack section titled 'Evidence Capture, Redaction, and GitHub Reporting'. It is designed for the Enterprise Security Assessment Lab GitHub portfolio and describes the objective, tooling, commands, sanitized output, evidence mapping, security interpretation, remediation, and redaction requirements for this specific section.

This document is written as a public portfolio version. It does not contain live client data, production system details, unredacted credentials, session cookies, raw hash dumps, or destructive attack instructions.

Field	Details
Project Area	Web Application Vulnerability Assessment and Penetration Testing
Primary Target	DVWA running in a local lab environment
Testing Environment	Kali Linux attack machine with Docker-based vulnerable target
Testing Style	Reconnaissance -> Enumeration -> Manual validation -> Exploitation proof -> Evidence capture
Methodology Reference	OWASP Web Security Testing methodology, OWASP Top 10, and PTES-style workflow
Prepared By	Jagriti Banerjee

2. Section Overview

Item	Details
Section Number	13
Section Title	Web VAPT Section 13 - Evidence Capture, Redaction, and GitHub Reporting
Risk / Severity Style	Reporting and Safety Control
OWASP Mapping	Supports responsible disclosure, privacy protection, and safe security documentation.
Primary Evidence Count	2

2.1 Objectives

- Map every finding to screenshot evidence.
- Ensure evidence is clear enough for recruiters and reviewers.
- Prevent accidental publication of credentials, hashes, cookies, and shell details.
- Maintain professional GitHub documentation standards.

2.2 Tools Used

Tool	Purpose / Use in this section
Flameshot/Screenshot tool	Capturing portfolio evidence.
Markdown	GitHub-ready documentation.
GitHub	Public portfolio hosting with redaction controls.
PDF reports	Professional documentation deliverables.
.gitignore	Prevents committing sensitive raw files.

3. Methodology and Execution Workflow

The execution workflow follows a controlled lab approach: confirm authorization and target scope, run only the tools required for the section, save outputs, validate observations manually where applicable, capture screenshots, redact sensitive values, and document findings without exaggeration.

3.1 Command Reference

```
# Safe pre-commit review workflow
git status
git diff --cached

# Search staged files for sensitive words
git diff --cached | grep -iE "password|secret|token|cookie|PHPSESSID|hash|credential"

# If a sensitive file is staged
git restore --staged path/to/sensitive-file
```

3.2 Expected / Sanitized Output

All evidence is mapped to a testing phase.
Sensitive values are redacted before upload.
Raw exploit files, dumps, cookies, and hashes are not committed.
GitHub documentation remains professional and safe.

3.3 Validation Criteria

- The target must be the local DVWA or authorized lab environment only.
- The output must match the intended section objective.
- Evidence screenshots must clearly show the validation result or tool output.
- Any sensitive values must be blurred, cropped, removed, or replaced with placeholders.
- The section must not claim findings that are not supported by evidence.

4. Evidence Mapping

Evidence File	Evidence Type / Reason
09-recon-report-files-generated.png	Recon report/output generation proof.
all Web VAPT screenshot evidence files	Full screenshot set used for final portfolio and reporting verification.

5. Professional Security Analysis

Strong evidence handling separates a professional portfolio from a risky one. Cybersecurity projects must prove practical work while avoiding secret exposure.

The uploaded Web VAPT scope emphasizes screenshot-based GitHub documentation and redaction of cookies, hashes, credentials, session IDs, and shell details.

Each individual PDF should reference evidence filenames rather than embedding unredacted secrets.

5.1 Why This Section Matters

This section matters because professional VAPT is not only about running a tool. It requires a clear objective, controlled execution, evidence capture, correct interpretation, and practical remediation. For recruiters and reviewers, this PDF shows that the work was structured, documented, and aligned with real security methodology.

5.2 Common Mistakes to Avoid

- Uploading raw outputs that expose cookies, hashes, paths, or credentials.
- Claiming production-level impact without production context.
- Reporting scanner output as a confirmed vulnerability without validation.
- Using unclear screenshots that do not show the relevant result.
- Mixing unrelated phases in one evidence file without explanation.

6. Risk Analysis

- Accidental credential leak through screenshots.
- Publishing raw hashes or cracked passwords.
- Exposing session cookies.
- Committing shell files or SQLMap dump folders.
- Making claims unsupported by screenshots.

6.1 Real-World Impact Statement

The real-world impact depends on the affected asset, authentication context, data sensitivity, privilege level, exposed functionality, and compensating controls. Because this project uses DVWA in a lab, severity is described as a real-world context estimate rather than a formal client CVSS score.

7. Remediation and Hardening Guidance

- Blur or crop sensitive values.
- Use .gitignore to block dangerous files.
- Use sanitized placeholders such as <REDACTED_COOKIE> and <REDACTED_HASH>.
- Review git status and staged diffs before every push.
- Document only what was actually completed.

7.1 Verification After Remediation

- 1 Re-run the original test case in the lab or development environment.
- 2 Confirm that the previous vulnerable behaviour no longer appears.
- 3 Check server logs for blocked or rejected attempts.
- 4 Capture new evidence showing the fix or defensive control.
- 5 Update the report status from validated/open to fixed/retested if applicable.

8. GitHub Redaction and Publishing Checklist

Cookies, hashes, credentials, session IDs, shell paths, database output, and private details must be redacted before public upload.

Item	Action Before Upload
PHPSESSID / Cookies	Blur or replace with <REDACTED_COOKIE>.
Password hashes	Blur or replace with <REDACTED_HASH>.
Cracked passwords	Do not publish; replace with <REDACTED_PASSWORD>.
Database dumps	Crop, blur, or summarize instead of uploading raw data.
Webshell paths/files	Do not upload shell source files; redact paths if needed.
Local usernames/paths	Blur if private or unnecessary.
Tool raw outputs	Upload only sanitized outputs or summary documents.

8.1 Safe Git Commands Before Push

```
git status
git diff --cached
git diff --cached | grep -iE "password|secret|token|cookie|PHPSESSID|hash|credential"
```

9. Conclusion

This document completes the individual Web VAPT PDF for Section 13. It documents the section objective, execution workflow, evidence mapping, professional interpretation, risk, remediation, and GitHub safety checks. The content is aligned to the actual Enterprise Security Assessment Lab Web VAPT evidence set and avoids unsupported production claims.

Legal and ethical notice: This documentation is for authorized lab testing only. Do not apply these techniques to public or third-party systems without explicit written permission.